Shrink First, Adapt Cautiously: A Training-Free Hierarchical Predictor with Leave-One-Out-Gated Test-Time Adaptation for the Predictive AI Evaluation Competition

[Author Name]
[Affiliation]
neelsirivara@outlook.com

Abstract

The Predictive AI Evaluation Competition (PAIEC) asks for the probability that an AI system (the *subject*) answers a benchmark *item* correctly, given the subject's visible attributes, the item text, and between 0 and 31 labels acquired on the same subject–benchmark pair. Scoring is the Brier area-under-the-learning-curve (ALC), which places 60% of its weight on budgets of at most seven labels. We describe a submission built without the official training release (`measurement-db` is gated and we had no credentials), i.e. a predictor with no fitted parameters: a hand-set attribute prior at budget zero, beta-binomial shrinkage of the pair success rate towards that prior, and two item-level adaptation components—a ridge-logistic adjustment on cheap item features and a TF-IDF nearest-neighbour correction—that are switched on only when a leave-one-out test on the acquired labels shows a clear improvement. A streaming acquisition policy removes near-duplicate candidates. Because the test data are private, we reproduce the official protocol locally on (i) a real testbed of 15 subject–benchmark pairs generated with four small open models (Qwen2.5 1.5B/3B/7B, Llama 3.2 3B) on GSM8K, SVAMP and BIG-Bench-Hard, and (ii) a synthetic two-parameter IRT testbed with oracle bounds. The shipped predictor reaches a Brier ALC of 0.2183 (95% CI 0.1923–0.2410) on the real testbed against 0.2745 for the organisers' empirical-mean baseline and 0.25 for the constant predictor, and 0.1797 against 0.2208 on the synthetic testbed. Oracle analysis splits the remaining headroom into two comparable parts—rate estimation at small budgets, which only a better prior can reduce, and item-level discrimination, which 31 labels cannot deliver: the gated item-level components change the ALC by +0.0000 (synthetic) and -0.0004 (real) relative to shrinkage alone, sentence embeddings did not beat TF-IDF, and uncertainty sampling did not beat random acquisition. The predictor is pure standard-library Python, deterministic, and runs at about 499 calls per second on one CPU core; the submission ZIP passes the organisers' validator. Code and text were produced with the help of an LLM coding assistant (Claude).

1 Introduction

Evaluating a new AI system on the full set of available benchmarks is expensive, and the number of benchmarks keeps growing [Liang et al., 2023]. A line of recent work therefore asks how well performance can be *predicted* rather than measured: from a few hundred carefully chosen items [Polo et al., 2024, Perlitz et al., 2024], from the scores of other models [Ruan et al., 2024], or from a handful of labelled instances of the new model itself [Pacchiardi et al., 2024]. PAIEC [Stanford AIMS Lab,

Technical report, Predictive AI Evaluation Competition (PAIEC), 40th Conference on Neural Information Processing Systems (NeurIPS 2026).

2026] turns this into a forecasting task with a precise interface: a function `predict([subject, item], labeled)` must return $\Pr(\text{correct})$ for one subject–item pair, where `labeled` contains the labels already acquired on the same subject–benchmark pair at budgets $n \in \{0, 1, 3, 7, 15, 31\}$. Pairs are scored by the Brier score [Brier, 1950] at each budget and aggregated into a Brier ALC with weights $(0.1, 0.2, 0.2, 0.2, 0.2, 0.1)$—the trapezoidal area under the Brier-versus-$\log_2(n{+}1)$ curve.

Two facts about this metric shaped our design. First, 60% of the weight sits at $n \leq 7$, where only the pair-level success rate can be estimated with any precision; item-level discrimination can pay off only at $n = 15$ and $n = 31$, which together carry 30%. Second, the organisers' empirical-mean baseline is *worse* than the constant 0.5 at $n = 1$ (its prediction is 0 or 1), so the single most valuable ingredient is a well-shrunk estimate of the pair rate. Our predictor therefore *shrinks first*: an attribute-based prior at $n = 0$, a beta-binomial posterior mean of the pair rate for $n \geq 1$ [Efron and Morris, 1975], and only then item-level adaptation, which is *gated* by a leave-one-out (LOO) test on the acquired labels so that it is applied only when the pair's own labels show that it helps.

We also faced a constraint that defines the submission: the official training release (`aims-foundations/measurement-db`) is gated behind a Hugging Face login, and the machine used for this work had no credentials; we deliberately did not create any account. The predictor therefore contains *no parameter fitted on any response data*. Every constant is a documented prior (Section 4), and the report tests their sensitivity. We see this as a strong, fully reproducible baseline for the competition and as a measurement of how far test-time adaptation alone can go.

Contributions. (1) A standard-library-only predictor and streaming acquisition policy that pass the official validator and run at hundreds of calls per second. (2) A local reproduction of the evaluation protocol—pairs, 50/50 acquisition/evaluation pools, a fixed random stream, nested budgets, Brier ALC and ECE—on a *real* testbed of 15 pairs built with local open models and on a synthetic IRT testbed with known item probabilities. (3) An analysis with paired bootstrap confidence intervals of what helps (prior, shrinkage), what helps marginally (item-level components), and what does not (sentence embeddings over TF-IDF, uncertainty sampling over random acquisition), together with oracle bounds that locate the remaining headroom.

2 Task, interface and metric

A *subject* is a dictionary of strings (`normalized_name`, `provider`, `release_date`, `access_date`, `harness`, `harness_version`, `reasoning_effort`, `subject_features_extra`); an *item* has `item_content` (the task text), `item_features` (e.g. `tier=medium;points=1`), `interactors`, and an anonymous `benchmark_id` shared by all items of a benchmark. Missing values are empty strings. For every subject–benchmark pair with at least 80 distinct items, the evaluator splits the items 50/50 into an acquisition pool and an evaluation pool; acquisition items are never scored. Candidates from the acquisition pool arrive in a fixed random order and an optional `acquisition_function(input, prediction, labeled, context)` returns `True` to reveal a label (31 at most); without it the evaluator samples labels uniformly. The nested label sets after 0, 1, 3, 7, 15 and 31 reveals are passed to `predict` for every evaluation item, giving Brier scores $B_0, \ldots, B_{31}$ and

$$\text{ALC} = 0.1\,B_0 + 0.2\,B_1 + 0.2\,B_3 + 0.2\,B_7 + 0.2\,B_{15} + 0.1\,B_{31}, \tag{1}$$

averaged with equal weight over pairs (lower is better). Expected calibration error (ECE, ten equal-width bins) is reported as a diagnostic [Guo et al., 2017]. Test benchmarks are disjoint from the training benchmarks. The evaluation runs offline (no package installation, no model downloads at run time, 8 hours per run).

3 Data

What we could and could not use. The official training data (`measurement-db`: item-level responses of many subjects on curated benchmarks) require accepting terms with a Hugging Face account; without credentials we could not download them, and we did not try to obtain them in any other way. We also inspected the organisers' inventory of eligible benchmarks (1,261 entries with slug, authors, code and data URLs and venue, mostly NeurIPS Datasets and Benchmarks, ICLR and ICML papers from 2024–2026) but did not fit anything on any of them. Consequently, *no training*

data of any kind entered the shipped predictor. The two local testbeds below were used only to evaluate design choices and to report the numbers in this paper; the handful of constants whose values were influenced by them are identified in Section 4 and probed by sensitivity analyses in Section 7.

Real Ollama testbed. To obtain item-level correctness labels of real language models we ran four small open-weight models locally with Ollama (Q4_K_M quantisation, greedy decoding, CPU only): Qwen2.5-Instruct 1.5B, 3B and 7B and Llama-3.2-3B-Instruct. Three public benchmarks were used, none of which appears in the PAIEC inventory (only a derived variant, `mr_gsm8k`, does): the first 200 GSM8K test items, the first 200 SVAMP test items (with the SVAMP problem type as `item_features`), and a 300-item mix of ten BIG-Bench-Hard subtasks (30 items each, subtask name as `item_features`). The 7B model was run on SVAMP only and its run was cut short at 126 items because the shared Ollama server was saturated by other jobs; a pair needs 80 items, so it is kept. Two harnesses are represented: a direct-answer prompt ("reply with only the final answer"; exact match after numeric or option normalisation; the three smaller models on all three benchmarks) and, for GSM8K/SVAMP, an earlier 8-shot chain-of-thought run of the Qwen models whose greedy answers we re-used. Each (model, harness, benchmark) triple with at least 80 items is one pair, giving 15 pairs, 3,226 labelled items, and success rates from 8.0% to 79.0% (Table 3 in the appendix). The subject dictionaries carry the real model names, providers, release dates and harness names so that the budget-0 prior is exercised on realistic inputs. Generation took about two hours of wall-clock time on a shared CPU; the raw records (prompt, raw answer, normalised answer, label) are released with the code.

Synthetic IRT testbed. Because the real testbed is small (small models, three benchmarks), we complement it with a generator that follows a two-parameter item response model [Lord, 1980, Embretson and Reise, 2000]. Subject s has ability θ_s correlated with its visible attributes (release year, reasoning effort, parameter count) plus noise; benchmark b has a location μ_b, a discrimination a_b and $K = 5$ topic clusters with difficulty offsets $c_k \sim \mathcal{N}(0, \sigma^2_{\text{topic}})$; item i in cluster $k(i)$ has difficulty $b_i = \mu_b + c_{k(i)} + \gamma\, z_{\text{len}}(i) + \varepsilon_i$ and $\Pr(y_{si} = 1) = \sigma(a_b(\theta_s - b_i))$. Item texts mix cluster-specific vocabulary (35% of tokens), benchmark template words, ambient vocabulary and numbers, so that lexical similarity carries difficulty information exactly when $\sigma_{\text{topic}} > 0$; half of the benchmarks expose a `tier=easy/medium/hard` feature derived from difficulty terciles. The main setting uses 10 subjects $\times$ 16 benchmarks $= 160$ pairs with 100–240 items each, $\sigma_{\text{topic}} = 0.70$ and $\gamma = 0.3$; a sweep varies both. Because the true probabilities are known, we can report two oracle references: the Brier score of the true item probabilities (the floor of any predictor) and that of the evaluation-set success rate used as a constant (the floor of any predictor without item-level information).

4 Method

All constants below are hand-set and shipped in `model.py`; the few whose values were adjusted after looking at the local testbeds are marked † and have sensitivity analyses in Section 7.

Budget-0 prior. A logistic prior $p_0 = \sigma(z)$ with $z = \text{clip}(z_{\text{subj}} + z_{\text{item}}, \pm 1.5)$, so $p_0 \in [0.18, 0.82]$. The subject part is $z_{\text{subj}} = 0.7 \cdot (0.35\,\text{clip}(t - 2024.5, \pm 2) + e + 0.25\,\text{clip}(\log_2(N/30\text{B}), \pm 4) + 0.1\,\mathbb{1}[\text{non-default harness}])$, where t is the release year (fractional), $e \in \{0, 0.15, 0.3, 0.45\}$ encodes an empty, low, other or high `reasoning_effort`, and N is a parameter count parsed from the name ("7b", "70B"); when no count is present, name tokens such as *mini/nano/flash* contribute -0.3 and *pro/opus/ultra* contribute 0.2. The item part parses `key=value` pairs in `item_features`: a tier/difficulty/level value matching *hard/expert/…* adds -0.6, *easy/basic/…* adds 0.5, a numeric level $\ell \in [1, 5]$ adds $-0.25(\ell - 3)$, and points above one subtract 0.1 each (capped at five); finally the item length adds $-0.12\,\text{clip}(\log_2(L/600), \pm 3)$ for L characters. The directions encode common knowledge (newer, larger, reasoning-enabled subjects do better; harder, longer, higher-value items are failed more often); the magnitudes were chosen to move the prior by at most a few tenths of a logit per attribute and were not tuned on either testbed.

Pair-level shrinkage. Given n acquired labels $y_1, \ldots, y_n$ of the same pair with item logits $z^{(j)}_{\text{item}}$ and mean $\bar{z}$, the prior rate is $r = \sigma(\text{clip}(z_{\text{subj}} + \bar{z}))$ and the posterior mean of a Beta–binomial model with prior strength $\alpha_0 = 4^\dagger$ pseudo-counts is $\rho = (\alpha_0 r + \sum_j y_j)/(\alpha_0 + n)$ [Efron and Morris, 1975].

The pair estimate for a target item is $p_{\text{pair}} = \sigma(\text{logit}\,\rho + z_{\text{item}} - \bar{z})$, i.e. the item cues shift the pair rate on the logit scale.

Item-level component 1: ridge features. From $n \geq 7$ labels, three cheap features of the item text—log length, digit density and log(1+number of sentences)—are standardised over the acquired items and a logistic regression with offset $\text{logit}\,p_{\text{pair}}$ and ridge penalty $\lambda = 4$ is fitted by six Newton steps; its adjustment $\text{clip}(\beta^\top \tilde{f}, \pm 1)$ is added to the target's logit.

Item-level component 2: TF-IDF nearest neighbours. Item texts (content, features and interactors, truncated to 4000 characters) are tokenised into lower-case words and numbers; the acquired set defines the vocabulary and document frequencies, with sublinear term frequency, smoothed idf and ℓ_2 normalisation (bigram weight 1; target terms absent from the acquired vocabulary are dropped: no). With cosine similarities s_j between the target and acquired item j and weights $w_j = (\kappa s_j)^2$, the correction is a shrunken kernel-weighted residual,

$$p = \text{clip}\Big(a + \frac{n}{n + N_0} \cdot \frac{\sum_j w_j\,(y_j - a)}{\tau + \sum_j w_j}\Big), \qquad N_0 = 4,\ \tau = 2, \tag{2}$$

where the anchor a is the pair estimate (optionally including the ridge adjustment). The default scale is $\kappa = 2$. With at most 31 short documents most bigrams are unique, so the bigram terms dominate the vector norms and cosine similarities are small (typically below 0.15 even for lexically related items); this deliberately keeps the correction small unless items share many terms. Section 7 compares this with a sharper unigram-only representation.

Leave-one-out gating. From $n \geq 5$ labels, every configuration in $\{\text{ridge off, on}\} \times \{\kappa \in \{0, 1, 2, 4\}\}$ ($\kappa = 0$ means no kNN) is scored by its leave-one-out Brier score on the acquired labels themselves: label j is predicted from the other $n - 1$ labels, including a leave-one-out shrinkage estimate and a leave-one-out ridge fit. The best configuration replaces plain shrinkage only if its mean per-item improvement exceeds 2 standard error(s) of that mean[†]; otherwise the predictor returns p_{pair}. This makes adaptation *opt-in per pair and budget*: a pair whose labels show no exploitable structure is predicted by shrinkage alone. Below 5 labels no item-level component is used at all (in preliminary runs an always-on kNN correction from one or three labels only added noise at B_1 and B_3, the budgets that carry most of the metric).

Acquisition policy. `labeling.py` implements the streaming interface and never raises. Candidates are accepted in stream order (which, being uniformly random, yields the same distribution of labels as the evaluator's default) except that a candidate whose maximum TF-IDF cosine similarity to an already acquired item exceeds 0.6 is skipped while the remaining budget is not urgent (fewer than 90% of the remaining candidates needed); it always accepts when the remaining candidates are needed to spend the budget and returns `False` once the budget is exhausted. We compare this *novelty* filter with the evaluator's random policy, with plain stream order, with the organisers' uncertainty band $|p - 0.5| \leq 0.15$ [Truong et al., 2026], and with a novelty-plus-uncertainty variant (skip candidates with $|p - 0.5| > 0.45$ once 7 labels are known).

Calibration and robustness. Predictions are clipped to $[0.01, 0.99]$. `predict` is wrapped in a fallback chain (full model → shrinkage only → smoothed label mean → 0.5), accepts labels given as booleans, numbers or strings, tolerates missing fields and non-dictionary inputs, and is deterministic.

5 Implementation

The submission ZIP (10.8 kB; members `README.md`, `labeling.py`, `model.py`) contains no `models.txt` and no `requirements.txt`: `model.py` imports only `math` and `re`, and `labeling.py` additionally `importlib`, `os` and `sys` to load its sibling `model.py` by path. It loads no files and makes no network calls, so the evaluator's offline policy is irrelevant to it. Two in-process caches (tokenised texts; statistics of each acquired set, including the LOO selection) only speed up repeated calls and never change a result. The organisers' `check_submission_zip.py` (commit `cc88a959`) passes on the ZIP; our own smoke test calls `predict` with the organisers' sample inputs at every budget, with and without `benchmark_id`, with malformed labels and inputs (34 cases) and checks

Table 1: Brier ALC (lower is better) with 95% bootstrap CIs over pairs and pooled ECE on the two local testbeds, all methods evaluated on identical random-policy acquisition trajectories. Oracles are references, not predictors.

	Synthetic IRT (160 pairs)		Real Ollama (15 pairs)	
Method	ALC [95% CI]	ECE	ALC [95% CI]	ECE
constant 0.5	0.2500 [0.2500, 0.2500]	0.111	0.2500 [0.2500, 0.2500]	0.135
empirical mean (organisers)	0.2208 [0.2055, 0.2362]	0.114	0.2745 [0.2302, 0.3117]	0.152
attribute prior only	0.2148 [0.2040, 0.2251]	0.041	0.2330 [0.2184, 0.2491]	0.071
prior + shrinkage	0.1797 [0.1691, 0.1900]	0.035	0.2187 [0.1925, 0.2416]	0.048
shrinkage + ridge (always on)	0.1814 [0.1708, 0.1918]	0.029	0.2162 [0.1919, 0.2367]	0.046
shrinkage + kNN (always on)	0.1794 [0.1687, 0.1895]	0.033	0.2189 [0.1924, 0.2420]	0.046
full: LOO-gated ridge + kNN (shipped)	0.1797 [0.1691, 0.1900]	0.032	0.2183 [0.1923, 0.2410]	0.046
oracle: pair success rate	0.1598 [0.1483, 0.1710]	0.000	0.1966 [0.1685, 0.2215]	0.000
oracle: true item probability	0.1379 [0.1280, 0.1482]	0.007	—	—

determinism. Mean latency is 4.85 ms per call on short items and 2.00 ms per call at budget 31 with 300-word items (maximum 54.8 ms when a new acquired set triggers the LOO selection), i.e. about 499 calls per second or 1.8 million calls per hour on one core of the development laptop—far below the 8-hour limit even for hundreds of thousands of evaluation calls.

6 Experimental setup

Our harness reproduces the protocol of Section 2: items of each pair are split 50/50 with a seeded shuffle, the acquisition pool is streamed in a seeded random order, the policy under test selects up to 31 labels, and the nested prefixes of that one trajectory are passed to every predictor for every evaluation item, so all methods are compared on identical labels. The evaluator's default policy is reproduced as accept-with-probability labels_remaining/items_remaining. We report Brier ALC with 95% bootstrap confidence intervals over pairs (1,000 resamples), paired bootstrap differences between methods (same pairs, same trajectories), per-budget Brier scores and pooled ECE. Baselines: the constant 0.5; the organisers' empirical mean (their code, which returns 0.5 without matching labels and the raw mean otherwise); our prior alone; shrinkage alone; shrinkage with each item-level component always on; and the shipped LOO-gated model. Everything runs on CPU; the full suite takes about an hour with four worker processes.

7 Results

Main comparison. Table 1 and Figure 1 give the headline numbers. On the real testbed the shipped predictor reaches 0.2183 [0.1923, 0.2410] against 0.2745 [0.2302, 0.3117] for the empirical mean and 0.25 for the constant; the paired difference to the empirical mean is -0.0562 [-0.0771, -0.0351] (significant; the full model is better on 80% of pairs) and to the constant -0.0317 [-0.0574, -0.0104] (significant). On the synthetic testbed the corresponding ALCs are 0.1797 [0.1691, 0.1900] versus 0.2208 and 0.25, with paired differences -0.0411 [-0.0488, -0.0339] and -0.0703 [-0.0805, -0.0592] (both significant). The empirical mean is *worse* than the constant at $n = 1$ on both testbeds ($B_1 = 0.330$ and 0.468) and only overtakes it from $n = 3$; on the real testbed its ALC (0.2745) is *above* the constant's 0.25 (paired difference +0.0245 [-0.0200, +0.0608], not significant), confirming the organisers' warning that an unsmoothed group mean can lose to 0.5.

Where the gain comes from. Decomposing the ablation rows of Table 1: the prior alone improves on the constant by -0.0352 (synthetic) and -0.0170 (real) ALC; shrinkage adds -0.0351 [-0.0415, -0.0285] and -0.0143 [-0.0307, +0.0002] on top (the real-testbed difference is not significant with 15 pairs). The item-level components are a much smaller effect. Against shrinkage, always-on kNN changes the ALC by -0.0003 [-0.0005, -0.0001] (synthetic) and +0.0002 [-0.0007, +0.0011] (real); always-on ridge by +0.0017 [+0.0011, +0.0024] and -0.0025 [-0.0062, +0.0015]; the LOO-gated full model by +0.0000 [-0.0002, +0.0003] (not significant) and -0.0004 [-0.0013, +0.0004] (not significant). At $n = 31$ alone the gated model changes the Brier score by -0.000 [-0.001, +0.000]

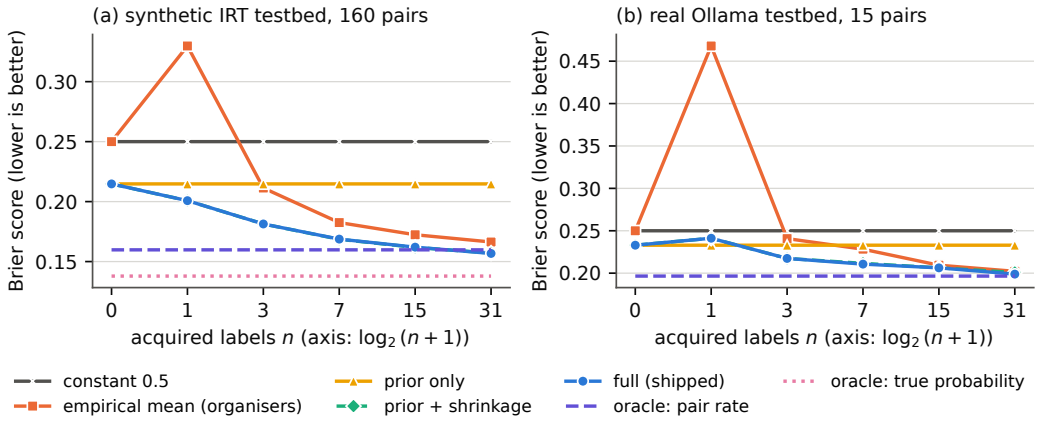


Figure 1: Brier score per budget (the ALC of Eq. 1 is the trapezoidal area under these curves on the $\log_2(n+1)$ axis). The empirical mean is worse than the constant at $n = 1$; shrinkage removes that penalty; the full model separates from shrinkage only at $n \geq 15$. Dashed/dotted lines are the oracle references.

(synthetic) and -0.002 [-0.010, +0.004] (real). We report these as what they are: item-level adaptation from at most 31 labels is a marginal gain whose sign has to be protected by the gate.

Headroom. The oracle rows explain why. On the synthetic testbed, knowing the true probability of every item gives an ALC of 0.1379, while knowing only the evaluation-set success rate gives 0.1598: perfect item-level discrimination is worth 0.0219 ALC, and it would have to be recovered from 15–31 labels at the two budgets that carry 30% of the weight. Shrinkage (0.1797) is 0.0199 above the rate oracle; that gap is the price of estimating a rate from few labels, concentrated at $n \leq 3$ (B_1: 0.041 above the oracle; B_{31}: -0.003), and it can only be reduced by a better prior, i.e. by training data. On the real testbed the rate oracle is 0.1966 against 0.2187 for shrinkage (gap 0.0221; B_1: 0.045, B_{31}: 0.004). The two parts of the headroom are of similar size, but they call for different remedies: the rate part needs a prior fitted to real subjects and benchmarks; the item part needs item difficulty information from outside the pair.

Prior strength. Figure 2(left) and the appendix give the ALC for $\alpha_0 \in \{1, 2, 4, 8\}$. The best value differs between testbeds (shrinkage: 2 on synthetic, 8 on real), reflecting the wider spread of pair rates in the synthetic generator; the ALC of the full model varies by 0.0038 (synthetic) and 0.0135 (real) over the range, so the choice matters at the third decimal. We ship $\alpha_0 = 4$, the value with the smallest worst-case regret over the two testbeds, and flag it as the constant most worth revisiting with access to real training data: α_0 should match the spread of success rates across the pairs actually evaluated.

When does text help? The synthetic sweep (Figure 2, right; Table 5) varies how much difficulty is explained by topic. With no topic or length signal ($\sigma_{\text{topic}} = 0.00$, $\gamma = 0.0$) the full model's paired difference to shrinkage is +0.0002 [-0.0000, +0.0005]—the gate's cost when there is nothing to find; at the strongest setting ($\sigma_{\text{topic}} = 1.00$) it is -0.0002 [-0.0005, +0.0001], while the item-level headroom (rate oracle minus true-probability oracle) grows from 0.0080 to 0.0306. The sign of the item-level effect does turn with the signal strength, but its size stays two orders of magnitude below the headroom: the gate protects against harm, it cannot create information.

Sentence embeddings versus TF-IDF. Replacing the TF-IDF representation by unit-normalised `sentence-transformers/all-MiniLM-L6-v2` embeddings (computed offline for the real testbed; the model is on the validator's allow-list via `models.txt`, so it could have been shipped) changes the full model's ALC from 0.2183 to 0.2181 (paired -0.0002 [-0.0005, +0.0000], not significant) and the always-on kNN variant from 0.2189 to 0.2206 (+0.0018 [-0.0008, +0.0047], not significant). With no measurable benefit and a real cost (a neural dependency, model loading and about 10 ms per embedding on CPU), we ship TF-IDF. Appendix A reports two further ablations: the LOO gate

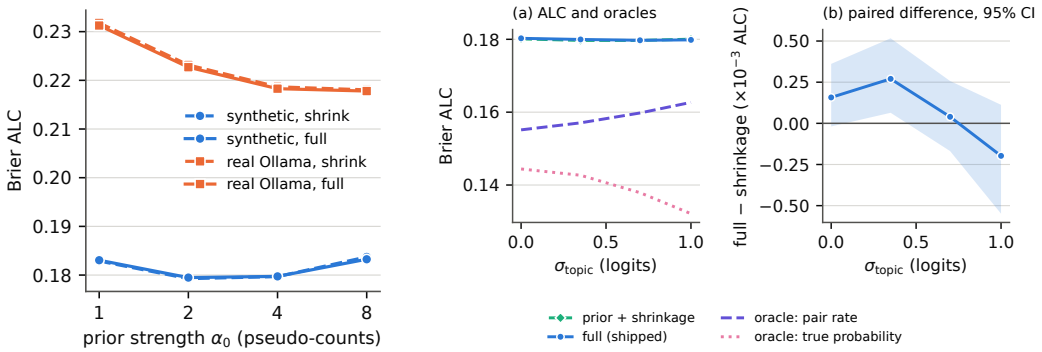


Figure 2: Left: Brier ALC as a function of the prior strength α_0 (dashed: shrinkage, solid: full model). Right: synthetic sweep of the topic-difficulty spread σ_{topic} at $\gamma = 0.3$: (a) ALC of shrinkage, the full model and the two oracles (the oracle gap is the item-level information that exists); (b) paired difference full − shrinkage with its 95% CI over pairs (negative favours the full model).

Table 2: Acquisition policies (full predictor): Brier ALC [95% CI] and paired difference to the evaluator's random policy on the same pairs (negative favours the policy). The BLE rule accepts a candidate when its current prediction satisfies $|p - 0.5| \leq 0.15$ [Truong et al., 2026].

	Synthetic		Real	
Policy	ALC [95% CI]	Δ vs random [95% CI]	ALC [95% CI]	Δ vs random [95% CI]
random (evaluator default)	0.1797 [0.1691, 0.1900]	—	0.2183 [0.1923, 0.2410]	—
stream order	0.1784 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2174 [0.1905, 0.2407]	-0.0009 [-0.0063, 0.0044]
novelty filter (shipped)	0.1784 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2172 [0.1902, 0.2410]	-0.0011 [-0.0067, 0.0048]
novelty + uncertainty	0.1785 [0.1675, 0.1888]	-0.0013 [-0.0038, 0.0013]	0.2172 [0.1902, 0.2410]	-0.0011 [-0.0067, 0.0048]
uncertainty band (BLE rule)	0.1798 [0.1693, 0.1896]	+0.0001 [-0.0027, 0.0030]	0.2156 [0.1882, 0.2391]	-0.0027 [-0.0086, 0.0026]

against always-on components and against its margin, and a sharper unigram TF-IDF, which separates topics better but makes the kNN worse.

Acquisition policies. Table 2 compares policies for the full predictor on identical pools and streams. Plain stream order differs from the evaluator's random policy by -0.0013 [-0.0038, +0.0013] (synthetic, not significant) and -0.0009 [-0.0063, +0.0044] (real, not significant)—both are uniform samples, so this is the trajectory-to-trajectory noise floor of the comparison. The shipped novelty filter gives -0.0013 [-0.0038, +0.0013] and -0.0011 [-0.0067, +0.0048]; neither testbed contains near-duplicate items at the 0.6 cosine threshold, so the filter never fires and the policy coincides with stream order (its value, if any, is on benchmarks with templated or repeated items). The organisers' uncertainty band gives +0.0001 [-0.0027, +0.0030] and -0.0027 [-0.0086, +0.0026], and the novelty-plus-uncertainty variant -0.0013 and -0.0011: with a predictor whose predictions are nearly constant within a pair at small budgets, selecting by $|p - 0.5|$ mostly reorders a random sample, and once item-level corrections exist it biases the sample towards items of intermediate predicted difficulty, which distorts the pair-rate estimate that carries most of the metric. None of the policies is significantly different from random on either testbed; we ship the one that is closest to random while removing redundant items.

Replication and calibration. Regenerating the synthetic testbed with two further seeds gives full-model ALCs of 0.2266 and 0.1963 (empirical mean 0.2737 and 0.2438; shrinkage 0.2264 and 0.1963). On the real testbed, four further acquisition trajectories give full-model ALCs between 0.2102 and 0.2208 (empirical mean 0.2469–0.2851), and the full model beats the empirical mean significantly in 4 of 4 trajectories. A 3×3 grid over the kNN scale and anchor ($\kappa \in \{1, 2, 4\}$, $\tau \in \{1, 2, 4\}$) spans 0.1806–0.1809 on a smaller synthetic generator, i.e. the kernel constants matter little. Pooled ECE (Table 1) is 0.032 (synthetic) and 0.046 (real) for the full model against 0.114 and 0.152 for the empirical mean and 0.111/0.135 for the constant; shrinkage towards a sensible prior is also what fixes calibration at small budgets.

8 Limitations

Local-versus-official gap. Our real testbed covers four small open models on three benchmarks with two harnesses; the official test set contains frontier systems on many held-out benchmarks. In particular the subject part of the prior (release date, reasoning effort, size) has essentially no variance on our testbed and is validated only on the synthetic generator, whose attribute–ability relation we chose ourselves. Absolute ALC values will differ; we expect the *ordering* shrinkage $>$ empirical mean $>$ constant at small budgets to transfer because it follows from the metric, but the sign and size of the item-level effects are testbed-dependent. *No training data.* With `measurement-db` the prior and α_0 could be fitted rather than set by hand; this is the obvious next step and would mostly affect B_0 and B_1. *Small real testbed.* 15 pairs give wide confidence intervals; several differences we report are not significant and are stated as such. *Stateless by design.* The predictor uses only the labels passed to the current call; it does not pool labels across subjects on the same benchmark, which the interface would in principle allow through `benchmark_id` but which we considered outside the spirit of the task and fragile under concurrent evaluation. *Weak text signal.* With at most 31 labels, lexical similarity recovers little of the available item-level headroom on either testbed; richer item models would need data beyond the pair. *Untested acquisition filter.* The novelty filter never fired on either testbed, so its effect on benchmarks with templated or repeated items is untested; on our testbeds it is exactly stream order.

9 Reproducibility

All code, results and this report are released at `[repository URL to be inserted by the author]` (the `paiec/` directory of the repository; also attached to the OpenReview submission). `experiments/src/model.py` and `labeling.py` are the shipped files; `build_submission.py` creates the ZIP and runs the organisers' validator; `smoke_test.py` checks the interface and timing; `real_testbed_generate.py` regenerates the Ollama testbed (needs Ollama with the four models and local copies of GSM8K, SVAMP and BBH); `simulate.py` is the IRT generator; `harness.py` the protocol; `run_experiments.py -suite all` and `run_extras.py` write every result JSON in `experiments/results/`; `build_paper.sh` runs `make_macros.py`, `make_figures.py` and `make_audit.py` (which re-derives every directional claim of this report from the JSON files) and compiles the PDF, so no number is typed by hand. All random seeds are fixed in the code; `predict` is deterministic. Hardware: one laptop CPU (8 cores, 34 GB RAM, no GPU).

10 Disclosure

Training data: none. The predictor has no fitted parameters; `measurement-db` was not accessed. *Evaluation data:* GSM8K and SVAMP test sets and ten BIG-Bench-Hard subtasks (local copies of the public Hugging Face datasets), used only to generate correctness labels of local models for the evaluation harness. *Pretrained models:* Qwen2.5-Instruct 1.5B/3B/7B and Llama-3.2-3B-Instruct via Ollama, as evaluation subjects only; `sentence-transformers/all-MiniLM-L6-v2` in one offline ablation, not shipped. No commercial API was called. *Compute:* a CPU-only laptop (8 cores, 34 GB RAM); about two hours for testbed generation (on a server shared with other jobs) and about one hour for the experiment suite. *LLM coding assistant:* the code, experiments and the text of this report were drafted by Anthropic's Claude (Claude Code) operating autonomously on the author's machine; the author reviewed the pipeline, is responsible for the content, and takes no credit for text that was generated.

References

Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950. doi: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319, 1975. doi: 10.1080/01621459.1975.10479864.

Susan E. Embretson and Steven P. Reise. *Item Response Theory for Psychologists*. Psychology Press, 2000. doi: 10.4324/9781410605269.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1706.04599.

Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023. arXiv:2211.09110.

Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Routledge, 1980. doi: 10.4324/9780203056615.

Lorenzo Pacchiardi, Lucy G. Cheke, and José Hernández-Orallo. 100 instances is all you need: Predicting the success of a new LLM on unseen data by testing on a few instances, 2024. arXiv:2409.03563; presented at the KDD 2024 Workshop on Evaluation and Trustworthiness of Generative AI Models.

Yotam Perlitz, Elron Bandel, Ariel Gera, Ofir Arviv, Liat Ein-Dor, Eyal Shnarch, Noam Slonim, Michal Shmueli-Scheuer, and Leshem Choshen. Efficient benchmarking of language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. arXiv:2308.11696.

Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks: Evaluating LLMs with fewer examples. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2402.14992.

Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.10938.

Stanford AIMS Lab. Predictive AI evaluation competition (NeurIPS 2026). Competition web page, 2026. URL `https://aims.stanford.edu/competition`. Accessed 2026-10-21.

Sang Truong, Wale Salaudeen, Nhi Truong, Serena Wang, Angelina Wang, Luke Guerdan, Ziang Xiao, Xing Xie, Nick Haber, and Sanmi Koyejo. PAIEC predictor baselines. GitHub repository, 2026. URL `https://github.com/aims-foundations/paiec_baseline`.

A Additional ablations

Gating. Replacing the LOO gate by always-on components at their default settings gives 0.1808 versus 0.1797 (synthetic; paired gated−always -0.0011 [-0.0017, -0.0005], significant) and 0.2169 versus 0.2183 (real; +0.0014 [-0.0019, +0.0045], not significant). The gate pays where always-on components overfit (the synthetic ridge) and costs a little where a component carries real signal (on the real testbed longer GSM8K problems are failed more often, which the always-on ridge exploits at $n \geq 15$); the second effect is within noise. Requiring 0, 1 or 2 standard errors of LOO improvement gives full-model ALCs of 0.1808, 0.1799 and 0.1797 (synthetic) and 0.2178, 0.2182 and 0.2183 (real); we ship the most conservative setting.

Sharper lexical similarity does not help. A unigram-only TF-IDF whose target terms are restricted to the acquired vocabulary separates same-topic from other-topic synthetic items far better (mean cosine about 0.3 versus 0.1 in the generator) but makes the always-on kNN *worse*: paired against the shipped representation, the always-on kNN changes by +0.0009 [-0.0001, +0.0020] (synthetic) and +0.0017 [+0.0001, +0.0031] (real), and the gated full model by +0.0004 [+0.0001, +0.0008] and +0.0001 [-0.0001, +0.0003]. With a handful of neighbours per topic the kernel-weighted residual is dominated by label noise; the bigram-diluted similarity acts as a regulariser. Item-level signal exists (the oracles show it) but 31 labels are too few to exploit it through similarity alone.

B Validator and smoke-test output

The organisers' validator exits with code 0 and prints

```
OK: paiec_submission.zip looks like a valid submission ZIP.
```

(full output in `experiments/results/validator_output.txt`; ZIP SHA-256 prefix `770f86e78358`). The smoke test (`experiments/results/smoke_test.json`) records 34 successful cases and the timings quoted in Section 5.

C Additional tables

Table 3: Real Ollama testbed: one pair per (model, harness, benchmark). Accuracy is the success rate over all items of the pair (acquisition and evaluation pools together).

Model (Ollama tag)	Harness	Benchmark	Items	Acc. (%)
qwen2.5:1.5b-instruct	8-shot CoT	GSM8K (test, first 200)	200	37.5
qwen2.5:3b-instruct	8-shot CoT	GSM8K (test, first 200)	200	24.0
qwen2.5:7b-instruct	8-shot CoT	GSM8K (test, first 200)	200	42.5
qwen2.5:1.5b-instruct	8-shot CoT	SVAMP (test, first 200)	200	79.0
qwen2.5:3b-instruct	8-shot CoT	SVAMP (test, first 200)	200	67.0
llama3.2:3b	direct answer	BBH mix (10 subtasks $\times$ 30)	300	49.3
qwen2.5:1.5b-instruct	direct answer	BBH mix (10 subtasks $\times$ 30)	300	39.3
qwen2.5:3b-instruct	direct answer	BBH mix (10 subtasks $\times$ 30)	300	29.3
llama3.2:3b	direct answer	GSM8K (test, first 200)	200	8.0
qwen2.5:1.5b-instruct	direct answer	GSM8K (test, first 200)	200	12.0
qwen2.5:3b-instruct	direct answer	GSM8K (test, first 200)	200	12.5
llama3.2:3b	direct answer	SVAMP (test, first 200)	200	33.0
qwen2.5:1.5b-instruct	direct answer	SVAMP (test, first 200)	200	32.5
qwen2.5:3b-instruct	direct answer	SVAMP (test, first 200)	200	39.5
qwen2.5:7b-instruct	direct answer	SVAMP (test, first 200)	126	64.3

Table 4: Brier score per budget for the main methods (random policy). The last column is the ALC of Eq. 1.

Testbed	Method	B_0	B_1	B_3	B_7	B_{15}	B_{31}	ALC
synthetic	constant 0.5	0.250	0.250	0.250	0.250	0.250	0.250	0.2500
	empirical mean	0.250	0.330	0.211	0.183	0.172	0.166	0.2208
	attribute prior only	0.215	0.215	0.215	0.215	0.215	0.215	0.2148
	prior + shrinkage	0.215	0.201	0.181	0.169	0.162	0.157	0.1797
	full (shipped)	0.215	0.201	0.181	0.169	0.162	0.157	0.1797
	oracle: pair success rate	0.160	0.160	0.160	0.160	0.160	0.160	0.1598
real	constant 0.5	0.250	0.250	0.250	0.250	0.250	0.250	0.2500
	empirical mean	0.250	0.468	0.241	0.228	0.209	0.202	0.2745
	attribute prior only	0.233	0.233	0.233	0.233	0.233	0.233	0.2330
	prior + shrinkage	0.233	0.241	0.217	0.212	0.206	0.201	0.2187
	full (shipped)	0.233	0.241	0.217	0.211	0.206	0.199	0.2183
	oracle: pair success rate	0.197	0.197	0.197	0.197	0.197	0.197	0.1966

Table 5: Synthetic signal sweep (160 pairs per setting, seed 0): ALC of shrinkage and of the full model, their paired difference, and the two oracle references.

σ_{topic}	γ	Shrinkage	Full [95% CI]	Full $-$ shrink [95% CI]	Oracle rate	Oracle p
0.00	0.0	0.1816	0.1818 [0.1708, 0.1926]	+0.0002 [-0.0000, 0.0005]	0.1543	0.1464
0.00	0.3	0.1801	0.1803 [0.1692, 0.1906]	+0.0002 [-0.0000, 0.0004]	0.1551	0.1444
0.35	0.3	0.1797	0.1800 [0.1689, 0.1905]	+0.0003 [0.0001, 0.0005]	0.1571	0.1427
0.70	0.3	0.1797	0.1797 [0.1691, 0.1900]	+0.0000 [-0.0002, 0.0003]	0.1598	0.1379
1.00	0.3	0.1800	0.1798 [0.1694, 0.1898]	-0.0002 [-0.0005, 0.0001]	0.1627	0.1321
0.70	0.0	0.1793	0.1796 [0.1688, 0.1896]	+0.0002 [0.0000, 0.0005]	0.1583	0.1392

Table 6: Sensitivity to the prior strength α_0 (Brier ALC).

	$\alpha_0 = 1$	$\alpha_0 = 2$	$\alpha_0 = 4$	$\alpha_0 = 8$
Synthetic, shrinkage	0.1829	0.1793	0.1797	0.1837
Synthetic, full	0.1831	0.1795	0.1797	0.1832
Real, shrinkage	0.2318	0.2232	0.2187	0.2180
Real, full	0.2313	0.2227	0.2183	0.2178